



Asynchronous JavaScript & Fetch/AJAX

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence



Session Outline

1. Why Asynchronous JavaScript?

2. Promise

3. `async` / `await`

4. Fetch API & AJAX

5. From JSON to the DOM

6. CORS

7. Worked Example & Labs



Learning Objectives

By the end of this session, you will be able to:

1. Explain why JavaScript needs an asynchronous mechanism, given it runs on a single thread.
2. Describe a Promise's three states and handle results with `.then()/.catch()`.
3. Write asynchronous code with `async/await`, including `try/catch` error handling.
4. Use the Fetch API to call an API, parse JSON, and render data onto the DOM.
5. Recognize a CORS error and explain, at a basic level, why it happens.



How to Approach This Session

- **Async is taught deeply only once** – right here, in JavaScript.
- When the backend moves to Python/FastAPI (`async def`, Week 6+), you'll only need new syntax – the thinking is identical.
- Today's end goal: **fetch real JSON data and render it onto a page**, without reloading.
- This is the pivot session – JSON is the bridge between Frontend and any Backend/AI model you build later.



Why Asynchronous JavaScript?



JavaScript Is Single-Threaded

- The browser has **one** main thread to both run JS and render the UI.
- If a slow task (e.g. a network call) runs **synchronously**, the whole page freezes (“blocks”) until it finishes.
- Solution: hand slow tasks to the browser to run in the background; JS keeps executing; results are handled once ready.
- This hand-off/resumption is coordinated by the **Event Loop**.



DS/AI Analogy: `fetch()` vs. `requests.get()`

- You already call APIs in Python with `requests.get()` – usually **synchronous/blocking**.
- A simple Python script can get away with blocking: there's no UI to freeze.
- `fetch()` in the browser is **non-blocking** by nature – the tab must stay responsive.
- **This is exactly why** JS needs Promise/async/await as core language features.



Promise

Promise – a Value That “Will Exist in the Future”



State	Meaning
pending	Still waiting for the operation to finish
fulfilled	Finished successfully, result available
rejected	Failed, error available



Handling a Promise: `.then()` / `.catch()`

```
1 fetch("https://jsonplaceholder.typicode.com/users")
2   .then(function (response) {
3     return response.json();
4   })
5   .then(function (data) {
6     console.log(data); // parsed array of users
7   })
8   .catch(function (error) {
9     console.error("Something went wrong:", error);
10  });
```

- `.then()` runs on fulfilled; `.catch()` runs on rejected.
- Chaining many `.then()` calls gets hard to read – `async/await` fixes this.



async / await

The `async` Keyword

```
1 async function greet() {  
2   return "Hello!"; // actually returns Promise<"Hello!">  
3 }
```

- Placing `async` before a function means it **always returns a Promise**.

The `await` Keyword

```
1 async function layDuLieu(url) {  
2   const response = await fetch(url);  
3   const data = await response.json();  
4   return data;  
5 }
```

- Valid only **inside an** `async function`.
- Pauses execution until the Promise settles, then continues – code reads top-to-bottom.
- Far more readable than chained `.then().then().then()`.

Error Handling with try/catch

```
1 async function layDuLieu(url) {  
2     try {  
3         const response = await fetch(url);  
4         const data = await response.json();  
5         return data;  
6     } catch (error) {  
7         console.error("Loi khi goi API:", error);  
8     }  
9 }
```

- Wrap await calls in try/catch to catch network or data errors.
- **Exam cue:** this *exact* pattern – async def + await + try/except – reappears in FastAPI (Week 6+).



Fetch API & AJAX

Fetch API – Calling the Network

```
1 async function layDuLieu(url) {  
2     try {  
3         const response = await fetch(url);  
4         const data = await response.json();  
5         return data;  
6     } catch (error) {  
7         console.error("Loi khi goi API:", error);  
8     }  
9 }
```

- `fetch(url)` returns a Promise resolving to a Response object.
- `.json()` (also async) parses the response body into a JS object/array.



What Is AJAX?

- **AJAX** = Asynchronous JavaScript And XML.
- Technique for requesting data from a server **without reloading the page**.
- Historically: XMLHttpRequest or jQuery Ajax helpers.
- Today: the **Fetch API** is the modern, native way – and JSON has replaced XML as the data format of choice.



Further Reading (Part 1)

Optional – self-study handout

- Ajax combines HTML, CSS, the DOM, and JavaScript (traditionally via XMLHttpRequest).
- Source: *Learning Web Design*, Ch. 22 – Using JavaScript and the DOM, “What Is Ajax?”



Lab 1: Promise Basics

Hands-on practice

- Simulate network delay with `setTimeout()` wrapped in a Promise.
- Call it once using `.then()`, and once using `async/await`.
- Compare readability between the two styles.



From JSON to the DOM



Converting Between JSON Text and JS Objects

- `JSON.parse(text)` – JSON string $\rightarrow$ JS object/array.
- `JSON.stringify(obj)` – JS object/array $\rightarrow$ JSON string.
- `response.json()` from Fetch already does the `JSON.parse()` step for you.

Rendering Fetched Data onto the Page

```
1 async function renderUsers(url) {  
2   const response = await fetch(url);  
3   const users = await response.json();  
4  
5   const table = document.querySelector("#user-table tbody");  
6   users.forEach(function (user) {  
7     const row = document.createElement("tr");  
8     row.innerHTML =  
9       ` ${user.id}</td><td>${user.name}</td>`; 10    table.appendChild(row); 11  }); 12 } |
```

- Reuses Week 4's DOM tools: createElement(), appendChild(), innerHTML.



Why This Is the Pivot Point of the Course

- Every backend prediction API you build later returns **JSON**.
- Every AI model output served over HTTP arrives as **JSON**.
- `fetch()` + DOM rendering is the mechanism that turns that JSON into something a user sees.



CORS



A Brief Introduction to CORS

- **CORS** (Cross-Origin Resource Sharing): a browser security policy.
- Calling an API on a **different domain** than your page → browser checks if that server allows it.
- If not allowed: request is blocked, error shown in DevTools Console – blocked by CORS policy.
- **For now:** just recognize this error. Server-side CORS configuration is covered with FastAPI (Week 10).

Further Reading (Part 2)

Optional – self-study handout

- MDN – *Using the Fetch API; Cross-Origin Resource Sharing (CORS)*.
- Source: *Learning Web Design*, Ch. 22, “Meet the APIs”.



Worked Example & Labs

Worked Example: Fetching and Rendering a Mock API



```
1 async function loadUsers() {
2   const response = await fetch(
3     "https://jsonplaceholder.typicode.com/users");
4   const users = await response.json();
5   const tbody = document.querySelector("#user-table tbody");
6
7   users.forEach(function (user) {
8     const row = document.createElement("tr");
9     row.innerHTML = `<td>${user.id}</td><td>${user.name}</td>`;
10    tbody.appendChild(row);
11  });
12 }
13 loadUsers();
```



Lab 2: Fetching Real Data

Hands-on practice

- Use `fetch()` to call a public/mock API (e.g. JSONPlaceholder “posts” or “users”).
- Loop through the JSON response and render it as an HTML `<table>` or list of cards.
- No page reload – the whole update happens via the DOM.



Lab 3: Project Integration

Hands-on practice

- Load data from a mock API (or a local `data.json` simulating an ML model's prediction output).
- Render it into your group project's frontend.
- Continues the running project thread from earlier weeks.



Review & Summary



Quick MCQ Review

1. Three states of a Promise? **pending, fulfilled, rejected**
2. Where can `await` be used? **Only inside an `async` function**
3. Method to convert a Fetch response into JS data? **`.json()`**
4. Structure to catch async errors? **`try {...} catch (error) {...}`**
5. Why does single-threaded JS need async? **To avoid blocking the UI on slow tasks**
6. When does a CORS error happen? **Browser blocks a request to a domain that didn't allow it**



Self-Check Checklist

- ☐ Can explain why JS needs asynchronous code in one sentence
- ☐ Know the three Promise states and `.then()/.catch()`
- ☐ Comfortable writing `async/await` with `try/catch`
- ☐ Can call `fetch()`, `parse .json()`, and render results to the DOM
- ☐ Recognize a CORS error in the Console

Summary and Next Session

- **JavaScript is single-threaded** – slow tasks must run asynchronously via the **Event Loop**.
- **Promise** represents a future value: pending → fulfilled/rejected.
- **async/await** lets async code read top-to-bottom; pair with try/catch.
- **Fetch API** (`fetch()` + `.json()`) is the modern way to do **AJAX**.
- Fetched JSON is rendered via Week 4's DOM methods; **CORS** governs cross-domain requests.
- Next: **Python & FastAPI basics** – the same `async/await` thinking, new syntax, now on the backend (Week 6).

Questions & Answers